CNC_Guard

CNC Guard: propuesta de análisis y alertas industriales

Elección

Kafka recibe telemetría sintética → NiFi valida y transforma → FastAPI sirve Random Forest → Elasticsearch indexa → Kibana visualiza.

Histórico y anomalías

Parquet + DuckDB para preparar datos; Isolation Forest como detector auxiliar de anomalías.

Compromiso

Construir y validar esta arquitectura con el stack del curso. Las alternativas se explican al final y no forman parte del despliegue propuesto.

Objetivo y alcance

Predecir riesgo de fallo a 24 horas como hipótesis de diseño

Negocio

50 CNC en Bilbao; el enunciado estima 1.200 €/hora de parada. La salida apoya la planificación de inspecciones.

Objetivo elegido

Para cada máquina y momento t: estimar fallo en las siguientes 24 horas con variables conocidas hasta t. Las 24 horas son una propuesta, no un requisito del PDF.

Condición de validez

Necesitamos historial longitudinal y averías fechadas. Con AI4I solo demostraremos clasificación de estado; no afirmaremos anticipación de 24 horas.

Qué hemos decidido

Una tecnología elegida para cada función

Flujo y servicio

Kafka recibe los eventos y NiFi ejecuta validación, transformación y llamadas HTTP a FastAPI. Random Forest clasifica en un servicio separado.

Datos y consulta

Parquet almacena el histórico; DuckDB consulta y prepara datasets; Elasticsearch indexa resultados; Kibana es el único panel operativo.

Fuera de la primera versión

No desplegamos Streamlit, MongoDB, InfluxDB, Spark, LLM ni actuadores industriales. FastAPI es el scoring; Isolation Forest ayuda en revisión, no veta alertas.

Por qué esta combinación

Cumplir el reto con tecnologías trabajadas en el curso

Integración y trazabilidad

NiFi y Kafka tienen prácticas del curso; permiten combinar fuentes, observar errores y reproducir eventos.

Modelos tabulares

Random Forest captura interacciones entre temperatura, torque, rpm y desgaste. Isolation Forest cubre el requisito de aprendizaje no supervisado sin exigir etiqueta en cada observación.

Histórico y operación

Parquet/DuckDB sirven al entrenamiento; Elasticsearch/Kibana sirven a consulta y visualización. Cada almacenamiento tiene una función distinta.

Arquitectura cerrada

El mismo recorrido para la demostración y la operación simulada

Entrada

CSV sintético → Kafka cnc-telemetry-csv → NiFi; registros inválidos a DLQ por event_id.

Inferencia

NiFi ConvertRecord y validación → InvokeHTTP /predict FastAPI → Elasticsearch y Kibana con niveles de alerta.

Histórico

Histórico Parquet y DuckDB; Random Forest entrenado offline en FastAPI; Isolation Forest como experimento independiente.

Fuentes y contrato de datos

Tres entradas fijadas para cubrir la integración

Telemetría CSV

machine_id, timestamp UTC, temperatura, rpm, torque, desgaste y quality_flag. No se inventa vibración si el dataset no la contiene.

Mantenimiento CSV

machine_id, fecha de intervención, tipo y fecha de avería. Es la fuente de contexto y de etiqueta futura cuando exista historial válido.

Logs de equipo

Conservar texto original, extraer máquina/fecha/código y asociarlo al contexto temporal. Cada evento lleva event_id y schema_version; entradas inválidas van a rechazo.

Python para todo el procesamiento

Un solo código de preparación e inferencia

Python + NumPy/pandas

Lenguaje elegido en el curso. NumPy realiza cálculo; pandas une tablas y calcula ventanas por máquina con datos del pasado.

scikit-learn

Pipeline de preparación, RandomForestClassifier e IsolationForest. Persistir solo modelos propios y registrar versión, variables y parámetros.

Estructura

Módulos de contratos, preparación, entrenamiento y servicio FastAPI; Jupyter contiene EDA. NiFi consume Kafka y solicita el scoring por HTTP.

NiFi: el flujo que construiremos

Validar antes de alimentar modelos

Configuración elegida

ConsumeKafka → ConvertRecord (CSVReader/JSON) → ValidateRecord, todo en laboratorio, nunca en CNC real.

Rutas fijas

Registros válidos → Elasticsearch y POST /predict; inválidos/422 a cuarentena; 5xx a reintentos acotados.

Evidencia requerida

Exportación NiFi, contadores, backpressure, replay y parada/reinicio; integración todavía por demostrar.

Kafka: topics y consumidores

NiFi consume Kafka y coordina las rutas de salida

Topics elegidos

Topic cnc-telemetry-csv, clave machine_id e identidad SHA-256 event_id por telemetría.

Grupos elegidos

NiFi group cnc-nifi-v1 consume Kafka, guarda telemetría y llama al scoring HTTP.

Garantía que probaremos

Entrega al menos una vez con deduplicación por event_id; sin prometer exactly-once.

Parquet: histórico elegido

Conservar entrenamiento y resultados fuera del índice operativo

Qué guardamos

Telemetría validada y predicciones en conjuntos separados; partición diaria y machine_id como columna. Conservar entradas originales y rechazos para auditoría.

Por qué Parquet

Formato columnar y comprimido tratado en ingeniería de datos. Adecuado para seleccionar columnas y preparar datasets históricos.

Política inicial

Propuesta de retención: 90 días de telemetría para la prueba longitudinal y todas las versiones del dataset evaluado. Dimensionar antes de fijar una política industrial.

DuckDB: preparación del dataset

SQL sobre el histórico Parquet

Consultas concretas

Seleccionar rango temporal y máquinas; unir intervenciones; obtener ventanas y preparar las particiones de entrenamiento, validación y test.

Por qué lo elegimos

Está tratado y ejercitado en el curso. Permite filtrar antes de pasar a pandas sin administrar un motor distribuido.

Límite asumido

Ejecuta en una máquina. Medir RAM, tiempo y disco; documentar riesgos de archivos distribuidos y réplicas sin afirmar que este diseño local sea un sistema distribuido.

Preparación elegida

Ventanas de 5 minutos y etiqueta a 24 horas

Limpieza

Separar errores de sensor de extremos plausibles; preservar flags y originales. Sin interpolación desde el futuro. Los faltantes se imputan con medianas aprendidas en train.

Variables iniciales

Valores actuales de temperatura, rpm, torque y desgaste; media, desviación y tendencia de los últimos 5 minutos por máquina. Ajustar unidades de la fuente.

Etiqueta y separación

y=1 si hay una avería fechada entre t y t+24 h. Excluir muestras sin seguimiento completo del horizonte; impedir que ventanas/etiquetas crucen indebidamente las particiones.

Tipo de IA elegido

IA débil basada en aprendizaje de datos

IA frente a ML

IA incluye razonamiento y aprendizaje. Nuestra aplicación usa ML: aprende de medidas e historial para una tarea definida.

Débil y fuerte

IA débil: tarea específica de riesgo CNC. IA fuerte: inteligencia general capaz de afrontar tareas amplias; no es la capacidad que proponemos ni demostramos.

Clasificación

Random Forest: ML supervisado tabular. Isolation Forest: ML no supervisado. Las reglas/difusa son IA simbólica y las redes neuronales son conexionistas; no se despliegan en esta propuesta.

Reglas para calidad y revisión

Reglas deterministas, sin motor difuso adicional

Reglas que sí usamos

Timestamp inválido, unidad desconocida o campo obligatorio ausente → rechazo. Dato demasiado antiguo → estado “sin dato actual”, nunca máquina sana por defecto.

Razón de la elección

Las reglas controlan el contrato del dato. Random Forest aprende relaciones con la etiqueta; Isolation Forest señala rareza. No necesitamos un tercer motor de riesgo para cubrir la propuesta.

Relación con la clase

La lógica difusa es una técnica pertinente y explicable, pero sus reglas y pertenencias añadirían una calibración que no requiere el núcleo elegido.

Random Forest: modelo principal

La alerta de riesgo sale de un único modelo supervisado

Qué aprende

Relación no lineal entre condiciones de funcionamiento, variables temporales y fallo dentro del horizonte. Usa etiquetas de avería cuando sean válidas.

Configuración inicial

200 árboles, max_depth=12, min_samples_leaf=5, class_weight=balanced y random_state=42. Son parámetros iniciales; la validación decide ajustes acotados.

Por qué lo elegimos

Modelo tabular desarrollado en el curso; maneja interacciones y no exige escalado como KNN/SVM. Registrar importancia de variables sin interpretarla como causalidad.

Isolation Forest: detector auxiliar

Las anomalías generan revisión, no una segunda decisión de fallo

Datos y ejecución

Entrenar con funcionamiento considerado normal cuando ese historial exista; aplicar a las mismas variables disponibles en inferencia, sin usar etiquetas futuras.

Configuración inicial

200 árboles, max_samples=256 y random_state=42. Guardar score; seleccionar umbral en validación y registrar alertas revisadas.

Función delimitada

Detectar patrones raros o regímenes nuevos. Una anomalía abre revisión técnica; no equivale a avería y no cancela la alerta supervisada.

Política de alertas elegida

Dos clases de aviso, sin consenso AND

Riesgo de fallo

Random Forest ≥ umbral validado → aviso de inspección planificable. Mostrar máquina, instante, horizonte y versión.

Anomalía

Isolation Forest supera su umbral → aviso de revisión de condición/dato, separado del riesgo supervisado. Mostrar score y motivo disponible.

Operación

Agrupar avisos persistentes por máquina para evitar repetir muestras como incidentes. El técnico confirma, descarta o programa; ninguna salida detiene físicamente una CNC.

Validación que aplicaremos

Mismo problema, particiones y preparación

Particiones elegidas

Historial real: 70% inicial train, 15% siguiente validation y 15% final test, con separación que evite fuga entre ventanas y horizonte. Registrar fechas y máquinas.

Umbral elegido

Elegir en validation el umbral que maximice F1 como criterio académico inicial. Informar recall, precision, falsos negativos y coste; no usar test para reajustar.

Prueba final

Evaluar una vez en test: matriz de confusión, F1, recall, antelación y falsas alarmas por máquina/día. Comparar con “sin fallo” y referencia logística, sin cambiar el modelo desplegado.

FastAPI: servicio de inferencia

Un modelo cargado una vez y consultado por NiFi

Al iniciar

FastAPI /health carga una sola vez el modelo propio con schema verificado y acceso local.

Por cada evento

NiFi InvokeHTTP /predict con sensores e historia causal de cada máquina; devuelve fallo, probabilidad y nivel.

Errores

422 a cuarentena, 5xx con reintentos limitados, cold start explícito y sin actuar sobre maquinaria.

Elasticsearch y Kibana

Una base de consulta y un único panel

Elasticsearch

Índice de predicciones con timestamp date, machine_id keyword, scores numéricos, calidad y model_version. Usar event_id como identificador para reintentos idempotentes.

Kibana

Data view sobre predicciones; filtros por máquina, fecha y tipo de aviso. Series temporales, ranking de avisos y vista de registros para investigar.

Por qué esta elección

Stack desarrollado en el curso: integra búsqueda, agregaciones y visualización. Retener inicialmente 30 días de resultados operativos; el histórico completo va a Parquet.

Panel que construiremos

Preguntas operativas para mantenimiento

Estado y evolución

Serie de temperatura/torque y riesgo por máquina; marcar intervenciones y huecos. ¿Cómo ha cambiado la condición?

Avisos pendientes

Ranking de máquinas por aviso de riesgo; lista separada de anomalías. ¿Qué inspeccionamos y qué dato necesitamos revisar?

Calidad y evaluación

Edad del último dato, rechazados y retraso; dispersión rpm–torque para contexto. Mostrar matriz de confusión y alertas confirmadas en la vista de evaluación.

Seguridad e integridad del diseño

Una versión local reproducible antes de cualquier conexión OT

Datos íntegros

Esquema y hashes de archivos; recuentos de ingesta/archivo/indexación; idempotencia por event_id. Probar duplicados, recuperación y corrupción detectable.

Laboratorio

Docker Compose local para NiFi, Kafka, Elasticsearch y Kibana. Volúmenes separados; acceso limitado al host. Un host único no aporta alta disponibilidad real.

Riesgos humanos

No incluir identificadores de operarios innecesarios; acceso y retención definidos. Analizar daño de falsas alarmas y fallos omitidos y sesgos por régimen/tipo de máquina.

Dimensionamiento inicial

Supuestos concretos para medir la propuesta

Carga de laboratorio

Simular 50 máquinas a 1 registro/s: 4,32 millones de eventos/día. A 200 bytes/evento son 864 MB/día sin overhead; 90 días: 77,76 GB sin compresión.

Pruebas de capacidad

Medir throughput, latencia de inferencia, lag de consumidores, RAM y disco. Registrar compresión de Parquet y tamaño real de índices; no extrapolar desde una muestra sin medir.

Coste y valor

Horas evitadas × 1.200 € menos falsas alarmas, inspecciones e infraestructura. La presentación no atribuye ahorro real ni rendimiento industrial a la simulación.

Alternativas viables: solo si cambia la necesidad

No forman parte de la primera versión elegida

Datos y panel

InfluxDB/TimescaleDB + Grafana si predomina telemetría temporal; MongoDB si el documento heterogéneo es central. Ambos casos implican revisar el panel Kibana elegido.

Modelo y escala

Boosting si aporta mejora reproducible frente a Random Forest; Spark si una máquina incumple capacidad/tiempo. No se introducen sin esa evidencia.

Conexión industrial

OPC UA/MQTT para una CNC/gateway compatible: extensiones externas no encontradas en el curso local. Requieren pruebas autorizadas; el laboratorio continúa con archivos.

Primera entrega: datos y entrenamiento

Construir primero un modelo reproducible

Artefactos

Contrato JSON, entradas con procedencia, dataset Parquet y consulta DuckDB; notebook EDA y módulos de preparación/entrenamiento.

Modelos elegidos

Random Forest e Isolation Forest propios, artefactos con hash y parámetros. Informe de particiones y comparación con referencias, sin declarar un ganador industrial.

Aceptación

Código reproducible, sin fuga demostrada, métricas y errores explicados. Si solo hay AI4I/sintéticos, etiquetar expresamente la entrega como prueba de estado y pipeline.

Segunda entrega: recorrido completo

Demostrar recuperación y visualización

Artefactos

Process group NiFi exportado, topics y grupo consumidor Kafka, servicio FastAPI, mappings Elasticsearch y dashboard Kibana reproducibles.

Prueba normal

Una entrada válida produce registro histórico, predicción y visualización con el mismo event_id y versión. Medir tiempos en cada etapa.

Prueba de fallo

Campo inválido, mensaje duplicado, reinicio de consumidor y destino temporalmente caído. Comprobar rechazo, idempotencia, relectura y ausencia de pérdida explicada.

Punto de partida real

Reutilizar infraestructura de análisis, cambiar el núcleo de decisión

Disponible

El prototipo Mamdani + Isolation Forest sigue como comparación. En el PR hay código Random Forest y API con tests aislados; el recorrido completo todavía no está probado.

Cambio decidido

Random Forest será el modelo principal; Isolation Forest será auxiliar y separado. Eliminar AND del servicio nuevo; conservar el prototipo anterior como referencia histórica.

Pendiente

Falta evaluar datos longitudinales y ejecutar Kafka → NiFi → FastAPI → Elasticsearch/Kibana. No reutilizar métricas antiguas como rendimiento de la nueva versión.

Plan de construcción

Tres hitos y responsables por función

Hito 1: dato y modelo

Ingeniería prepara contratos, histórico y calidad; modelado prepara etiquetas, particiones, Random Forest e Isolation Forest.

Hito 2: integración

Integración configura Kafka y NiFi → FastAPI; visualización prepara índices y Kibana. Toda la cadena debe conservar event_id para auditoría.

Hito 3: defensa

Informe con criterios rojos, métricas, riesgos, coste y límites; ANEXO4 con esta propuesta y ANEXO1 con roles, acuerdos y seguimiento.

Propuesta que defenderemos

Una arquitectura, un modelo principal y un panel

Compromiso técnico

Kafka → NiFi → FastAPI (Python/scikit-learn, Random Forest) → Elasticsearch/Kibana. Isolation Forest queda auxiliar y Parquet/DuckDB para análisis.

Resultado esperado

Avisos trazables por máquina con instante, horizonte, calidad y versión; histórico reproducible y evidencia de evaluación e integración.

Límite de la defensa

Cobertura docente por diapositiva, implementación y evidencias separadas. La anticipación de averías necesita datos longitudinales y validación propia.

Anexo · cobertura de criterios rojos

Correspondencia exacta: bloque 1 de 3

AA · IE1 a,c

Principios y técnicas de IA. Diapositiva 13. Evidencia de ejecución pendiente cuando corresponda.

AA · IE2 b,g

Clasificación y adecuación de modelos. Diapositiva 13. Evidencia de ejecución pendiente cuando corresponda.

AA · IE6 a,b

Riesgos y privacidad. Diapositiva 22. Evidencia de ejecución pendiente cuando corresponda.

ML · IE1 a,b

IA débil/fuerte y relación IA–ML. Diapositiva 13. Evidencia de ejecución pendiente cuando corresponda.

ML · IE2 b,c

Tipos, usos, técnicas y herramientas ML. Diapositiva 13. Evidencia de ejecución pendiente cuando corresponda.

ML · IE3 c,f

Entrenamiento y predicción nueva. Diapositiva 15. Evidencia de ejecución pendiente cuando corresponda.

Anexo · cobertura de criterios rojos

Correspondencia exacta: bloque 2 de 3

ML · IE4 b,c

Caracterización y aplicación no supervisada. Diapositiva 16. Evidencia de ejecución pendiente cuando corresponda.

ML · IE6 a,b

Idoneidad y evaluación práctica. Diapositiva 18. Evidencia de ejecución pendiente cuando corresponda.

Programación · IE1 b,e

Lenguaje y estructura del programa. Diapositiva 7. Evidencia de ejecución pendiente cuando corresponda.

Programación · IE2 c,d

Definición e implementación de la aplicación. Diapositiva 19. Evidencia de ejecución pendiente cuando corresponda.

Sistemas Big Data · IE1 b,c

IE1/b → diapositiva 11; IE1/c → diapositiva 6. La ejecución sigue pendiente.

Sistemas Big Data · IE2 a,b

Clasificar gráficos y relacionarlos con objetivo/datos. Diapositiva 21. Evidencia de ejecución pendiente cuando corresponda.

Anexo · cobertura de criterios rojos

Correspondencia exacta: bloque 3 de 3

Sistemas Big Data · IE3 a,d

Extracción, almacenamiento y seguridad. Diapositiva 10. Evidencia de ejecución pendiente cuando corresponda.

Sistemas Big Data · IE4 e,f

Procesamiento automático y visualización. Diapositiva 20. Evidencia de ejecución pendiente cuando corresponda.

Big Data aplicada · IE1 b,d

IE1/b → diapositiva 8; IE1/d → diapositiva 11. Integración pendiente.

Big Data aplicada · IE3 a,b

Calidad y riesgos de integridad al distribuir. Diapositiva 22. Evidencia de ejecución pendiente cuando corresponda.

Big Data aplicada · IE5 a,b

IE5/a → diapositiva 6; IE5/b → diapositiva 12. Validación pendiente.

Anexo · fuentes y límites

Trazabilidad de la propuesta

Enunciado

1Erronka_ikaslearen_txostena. pp. 2–7: contexto y criterios; pp. 8–13: equipo, evaluación y planificación.

Curso

AA: conceptos/difusa/ética. ML: introducción, modelos, validación. Programación: datos, API y Streamlit. Big Data: ingeniería/Elastic. NiFi: prácticas/series. Kafka: básico/avanzado.

Distinción de fuentes

Las notas de cada diapositiva citan rutas locales y fuentes primarias externas. README del prototipo se usa para el estado existente, no como autoridad del temario.